

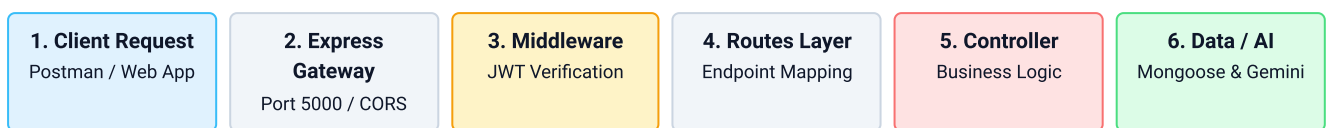
Phase 3: Project Design Phase

Project Name: **FitSense (FitTrack AI)** — System Architecture, Modules, & Relational Schemas

3.1 Architectural Foundation: Layered MVC Pattern

FitSense is engineered upon the classic **Model-View-Controller (MVC)** design pattern adapted for RESTful application programming interfaces. Because the system functions strictly as a headless API service, the conventional graphical "View" is decoupled into client consumers (Postman, Thunder Client, React frontends, mobile applications) communicating via standardized HTTP JSON payloads. Internal server logic separates routing boundaries, authorization middleware shields, domain controllers, and persistent database entities into distinct architectural tiers.

3.2 Architectural Flow & Layered Processing Pipeline



3.3 Detailed Subsystem Module Descriptions

1. Client Layer (API Consumers)

Entry interface consisting of front-end applications, testing tools (Postman, Thunder Client), or mobile clients.

- Constructs and transmits HTTP requests with query params and JSON bodies.
- Caches the stateless JWT bearer token received from login responses.
- Attaches the authorization token to headers of all secured requests.

2. Express Server Gateway

Central operational entry point running on Node.js v16+ listening on port 5000.

- Parses JSON payload bodies and URL-encoded query parameters.
- Configures Cross-Origin Resource Sharing (CORS) policies.
- Mounts application routers (/api/auth, /api/workouts, /api/ai).
- Provides global exception interception via centralized error handlers.

3. Authentication Middleware Shield

Security barrier protecting private user records and operational workout data.

- Extracts the `Authorization: Bearer <token>` string from request headers.
- Verifies token integrity and signature against `JWT_SECRET`.
- Decodes user payload and injects user identity into the request object (`req.user`).
- Halts unauthorized requests immediately with HTTP 401 Unauthorized status.

4. Controller & Service Layer

Executes business decisions, database mutations, and external API queries.

- Auth Controller:** Controls registration, login credential validation, and profile responses.
- Workout Controller:** Governs CRUD lifecycle and multi-parameter regex search filtering.
- Gemini AI Service:** Builds prompt context from user parameters and dispatches requests to Google Gemini.

3.4 Directory Structure and File System Layout

server/src/ contains: config/db.js (MongoDB connector), controllers/ (authController.js, workoutController.js, aiController.js) middleware/ (auth.js, errorHandler.js) models/ (User.js, Workout.js) routes/

3.5 Database & Data Model Design (MongoDB & Mongoose)

Data persistence utilizes MongoDB collections managed through strict Mongoose schemas. Document relationships implement a **One-to-Many (1:N)** structural model: one authenticated User creates multiple Workout documents.

User Collection Schema (`models/User.js`)

Attribute	BSON Type	Constraints & Rules
_id	ObjectId	Primary Key (Auto-generated)
name	String	Required, Trimmed whitespace
email	String	Required, Unique, Lowercase, Regex Pattern
password	String	Required, Min 6 chars, Bcrypt Hashed
createdAt	Date	Timestamp (Auto-managed)
updatedAt	Date	Timestamp (Auto-managed)

Hooks: Pre-save hook hashes password via `bcrypt.genSalt(10)`. Method `matchPassword()` performs hash comparison.

Workout Collection Schema (`models/Workout.js`)

Attribute	BSON Type	Constraints & Rules
_id	ObjectId	Primary Key (Auto-generated)
user	ObjectId	Foreign Key ref: 'User' (Required)
workoutName	String	Required, Trimmed whitespace
category	String	Enum: Cardio, Strength, Yoga, Running, Cycling, Walking
duration	Number	Required, Min: 1 minute
caloriesBurned	Number	Required, Min: 0 (non-negative)
workoutDate	Date	Default: Date.now
timestamps	Boolean	Auto-generates createdAt, updatedAt

3.6 Entity Relationship (ER) Structural Description

Derived from the project's documented ER architecture diagram, the full data model recognizes relational entities connecting users with logs, categories, and AI analytics:

- USER (1) – CREATES – (N) WORKOUT:** A single authenticated user owns zero to many workout records. The workout document preserves this link via the `user` foreign key field containing the user's unique `ObjectId`.
- USER (1) – USES – (N) AI_INTERACTION:** Users can invoke AI prompt generation multiple times, yielding historical records of prompt inputs and AI responses.
- WORKOUT_CATEGORY (1) – BELONGS_TO – (N) WORKOUT:** Workouts map to categorical classification entities (Cardio, Strength, Yoga, Running, Cycling, Walking).
- WORKOUT (1) – HAS – (N) EXERCISE:** Relational representation permitting workouts to break down into multi-set exercise records (sets, reps, duration).
- WORKOUT (1) – TRACKS – (N) PROGRESS:** Workouts correlate to longitudinal tracking data (body weight, body fat %, notes).

3.7 Data Flow & Algorithmic Execution Workflows

Workflow A: Authenticated Workout Ingestion

- Client submits `POST /api/workouts/` with workout fields and JWT Bearer header.
- Middleware decodes JWT, validates signature, and maps user ID to `req.user`.
- Workout Controller validates schema types (duration ≥ 1 , calories ≥ 0 , valid category).
- Controller constructs document binding user: `req.user.id`.

Workflow B: Google Gemini AI Prompt Pipeline

- Client submits demographic parameters (Age: 22, Goal: Weight Loss, Experience: Beginner).
- AI Controller extracts variables and calls `geminiService.js`.
- Service injects values into a structured fitness context prompt.
- Asynchronous HTTPS call sent to Google Gemini API using `GEMINI_API_KEY`.